

WIYSE SCHOOL

AI Engineering Assessment

Intelligent RAG System

Practical Assignment — Questions

Scope: Part 0 · Level 1 · Level 2.1 | Orchestration: LangGraph (required)

Name: _____ Date: _____

Overview

Build an AI-powered question-answering assistant. The system must answer natural-language questions by retrieving relevant information from a knowledge base you construct yourself, then generating grounded, accurate responses. You are free to choose any domain or dataset you find interesting — the quality of your pipeline matters far more than the topic you pick.

Required orchestration: implement your pipeline as a **LangGraph state graph** with a typed state that flows through the nodes (rewrite → retrieve → generate).

Deliverables: GitHub repository link + a written report (PDF or Markdown). Submit your GitHub link before the stated deadline.

Part 0 — Data & Chunking Strategy (Required)

0.1 — Choose Your Dataset

Describe the dataset you selected:

- Where does the data come from?
- What is the structure (plain text, JSON, PDFs, CSVs, web pages, etc.)?
- Roughly how many documents / records does it contain?

0.2 — Design Your Chunking Strategy

There is no predefined list to pick from. Study your data first, then decide how it should be split. Start by asking yourself:

- What is the natural unit of meaning in this data (a sentence, paragraph, record, section)?
- Does splitting in the wrong place destroy the answer to a question?
- How will a user phrase a query, and what chunk size gives the best chance of matching it?

Required in your report:

- Describe the chunking strategy you arrived at and how you defined it.
- Explain your reasoning: what did you observe about your data that led to this decision?
- What did you try or consider that you rejected, and why?
- What are the weaknesses of your approach, and how did you mitigate them (chunk size, overlap, metadata)?

Level 1 — Foundation (Mandatory)

Goal: build a working end-to-end RAG pipeline that can answer questions about your dataset.

1.1 — Data Ingestion

- Load and pre-process your chosen dataset.
- Apply your chosen chunking strategy.
- Store chunk-level metadata (e.g., source, section title, page number, date) alongside the text.

1.2 — Embedding & Indexing

- Embed all chunks using a sentence-transformer or API-based embedding model of your choice.
- Index the embeddings in a vector store (Qdrant, Chroma, FAISS, Pinecone, Weaviate — your choice).
- Justify your choice of embedding model in the report (multilingual support, dimension size, speed, cost).

1.3 — Basic Retrieval

- Given a user query, embed it and retrieve the top-k most similar chunks.
- Display the retrieved chunks with their similarity scores and source metadata.

1.4 — Answer Generation

- Pass the retrieved chunks as context to an LLM (any provider: OpenAI, Anthropic, Google, open-source).
- Generate a grounded answer that only uses the retrieved context — the model must not hallucinate facts not present in the chunks.
- Print the final answer alongside the sources it was derived from.

Deliverable: a script or notebook that takes a question as input and prints the answer + sources.

Level 2.1 — Query Rewriting

Goal: make the system smarter about understanding what the user is asking.

- Before retrieving, rewrite the user query to be cleaner, more specific, and in English (if your data is multilingual or the user might write in other languages).
- Use an LLM prompt for this.
- Show the original query and the rewritten version in your output.

Orchestration Requirement — LangGraph

- Implement the pipeline as a LangGraph state graph with nodes: rewrite → retrieve → generate.
- Define a typed state schema (TypedDict or Pydantic) that flows through all nodes.
- Include a diagram of your graph (hand-drawn or generated) in the report.

Evaluation Rubric

Component	Weight
Part 0: Data & chunking justification	20%
Level 1: Working RAG pipeline	50%
Level 2.1: Query rewriting	15%
LangGraph orchestration (typed state + graph diagram)	15%

Code quality, clarity, and the written report are assessed within each component.

Rules

- Python is strongly recommended. Any LLM provider or open-source model is allowed.
- Build the pipeline yourself — no end-to-end frameworks (Flowise, Dify, n8n) that hide the internals.
- Your report must explain every design decision, not just describe what you built.
- Academic integrity applies: the code must be your own work. Citing AI tools you used is encouraged, not penalised.

Submission Checklist

- GitHub repo with a clear README.md (setup instructions, how to run).
- All dependencies listed (requirements.txt or pyproject.toml).
- A .env.example file listing required keys (never commit real keys).
- Written report (PDF or Markdown) covering Part 0, Level 1, and Level 2.1.
- At least one end-to-end demo (script, notebook, or short screen recording).